Paper Title

Author 1
author1@email.com
Institution
City, State, Country

Author 2✉
author2@domain.com
Institution
City, State, Country

Abstract

We propose an accuracy-guided method for structural channel pruning in convolutional neural networks. The method learns a binary gate for each intermediate channel by combining a classification loss with a sparsity penalty and an optional entropy term. The sparsity penalty encourages fewer active channels, while the entropy term keeps gate probabilities from becoming deterministic too early. During training, we adjust the sparsity penalty using the validation accuracy gap relative to a dense reference model. The learned gate probabilities then guide permanent channel removal, and the resulting compact network is fine-tuned. Experiments with MobileNetV2 and ResNet-50 illustrate the trade-off between model size and classification accuracy.

CCS Concepts: • **Computing methodologies** → **Neural networks**; *Supervised learning*.

Keywords: Adaptive Computation, Variational Inference, Gated Networks, Gumbel-Softmax, Layer Pruning, Dynamic Inference

ACM Reference Format:
Author 1 and Author 2. 2026. Paper Title. In *Proceedings of Conference Title (Conference acronym 'XX)*. ACM, New York, NY, USA, 15 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Deploying convolutional neural networks under a limited parameter budget requires deciding which parts of the network can be removed while preserving predictive accuracy. Structural channel pruning addresses this problem by removing entire channels and the associated weights, yielding a narrower network. The central challenge is to determine how many channels can be removed and which ones to keep.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, City, Country
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/06
https://doi.org/XXXXXXX.XXXXXXX

Prior work approaches channel selection and conditional execution from different perspectives. AutoPruner learns binary masks for filter removal, and DepGraph identifies groups of parameters that must be pruned together to preserve architectural consistency [1, 10]. ConvNet-AIG uses binary gates to select which residual transformations to execute for each input [14]. We use binary gates during training to select intermediate channels for permanent removal. The resulting compact architecture is fixed across inputs.

Our method jointly learns the network weights and the opening probability of each channel gate. The training objective combines classification loss with a penalty on the expected number of active channels and an optional entropy term that encourages exploration during the search. When the entropy coefficient is positive, the two regularization terms admit an interpretation as a KL penalty toward a sparse Bernoulli prior. We adjust the sparsity coefficient using the validation accuracy gap to a dense reference model. Once the search is complete, the learned gate probabilities guide the construction of candidate subnetworks, and the selected subnetwork is fine-tuned after structural pruning.

Our contributions are threefold.

- We formulate channel selection with learnable Bernoulli gates and relate the entropy and sparsity terms to a sparse Bernoulli prior.
- We use validation feedback to adjust the sparsity penalty during the search, increasing it when the accuracy gap is small and relaxing it when the gap becomes too large.
- We turn learned gate probabilities into compact, fixed architectures and examine the trade-off between parameter count and test accuracy using iterative pruning on MobileNetV2 and one-shot pruning on ResNet-50.

2 Related Work

Feature selection and structured sparsification seek compact predictive models by selecting input variables, hidden units, or channels during training. Group sparse regularization penalizes groups of outgoing weights to jointly learn network weights, select input features, and determine hidden-layer widths [12]. Stochastic Gates (STG) uses clipped Gaussian gates to learn a subset of input features while penalizing the expected number of selected inputs [15]. At the weight level, stochastic gates also allow the expected number of nonzero parameters to be penalized through L_0 regularization [9].

Structural channel pruning applies selection to intermediate feature maps in convolutional networks. Network Slimming uses sparsity-regularized batch normalization scaling factors [8], while AutoPruner learns binary filter masks jointly with network weights [10]. Discrimination-aware Channel Pruning alternates greedy channel selection and parameter optimization using auxiliary classification and feature-reconstruction losses [16]. CHIP evaluates channel independence through the change in the nuclear norm of feature maps when a channel is removed [13]. DepGraph identifies groups of parameters that must be pruned together to preserve architectural consistency [1].

ConvNet-AIG uses input-dependent binary gates to select residual transformations at inference time [14]. Herrmann et al. learn channel gates using Gumbel-Softmax and a batch activation penalty that targets a prescribed activation rate, supporting both fixed pruning and input-dependent computation [4]. Our method also learns channel gates during training to obtain a fixed compact network. Its objective penalizes expected channel activity and optionally includes an entropy term that discourages gate probabilities from becoming deterministic early in the search.

The amount of pruning can also be adapted during optimization. DCP-Adapt determines the number of retained channels using a stopping criterion based on the improvement in its selection loss [16]. AMC uses reinforcement learning to search for layer-wise compression ratios [3]. Our method adjusts the strength of the sparsity penalty during training using the validation accuracy gap relative to a dense reference model. The learned gate probabilities then guide permanent channel removal, followed by fine-tuning of the compact network.

3 Methodology

3.1 Channel-Wise Gating

Consider a residual network with L blocks. For clarity, we describe a block with two transformations:

$$\mathbf{h}_l = a_{\theta,l}(\mathbf{x}_{l-1}), \qquad \mathbf{x}_l = s_l(\mathbf{x}_{l-1}) + b_{\theta,l}(\mathbf{h}_l), \tag{1}$$

where $a_{\theta,l}$ produces an intermediate feature map with C_l channels, $b_{\theta,l}$ maps these features to the residual output, and s_l is an identity shortcut or a projection.

We introduce one binary gate for each intermediate channel:

$$g_{l,c} \in \{0,1\}, \qquad c = 1, \ldots, C_l. \tag{2}$$

The gated block becomes

$$\boxed{\mathbf{x}_l = s_l(\mathbf{x}_{l-1}) + b_{\theta,l}\left(\mathbf{g}_l \odot a_{\theta,l}(\mathbf{x}_{l-1})\right),} \tag{3}$$

where $\mathbf{g}_l = (g_{l,1}, \ldots, g_{l,C_l})$ and $\odot$ denotes channel-wise multiplication broadcast over spatial dimensions.

When $g_{l,c} = 0$, intermediate channel c makes no contribution to the subsequent transformation. Because gating changes the intermediate representation rather than the block output dimensions, the residual addition remains dimensionally valid.

For a standard convolutional block, a permanently disabled intermediate channel can be removed by deleting its output filter from the preceding convolution, its associated channel-wise normalization parameters, and the corresponding input-channel slice of the following convolution.

Let

$$K = \sum_{l=1}^{L} C_l \tag{4}$$

be the total number of gated channels, and let $\mathbf{g} \in \{0,1\}^K$ collect all gate variables.

3.2 Probabilistic Formulation

For an input $\mathbf{x}$ and target $\mathbf{y}$, define

$$p_\theta(\mathbf{y}, \mathbf{g} \mid \mathbf{x}) = p_\theta(\mathbf{y} \mid \mathbf{x}, \mathbf{g}) p(\mathbf{g}), \tag{5}$$

where $p_\theta(\mathbf{y} \mid \mathbf{x}, \mathbf{g})$ is the predictive likelihood of the gated network and $p(\mathbf{g})$ is a prior over channel masks.

The marginal likelihood is

$$p_\theta(\mathbf{y} \mid \mathbf{x}) = \sum_{\mathbf{g} \in \{0,1\}^K} p_\theta(\mathbf{y} \mid \mathbf{x}, \mathbf{g}) p(\mathbf{g}). \tag{6}$$

Exact evaluation requires summing over exponentially many masks. We therefore introduce a variational distribution

$$q_\phi(\mathbf{g} \mid \mathbf{x}) = \prod_{l=1}^{L} \prod_{c=1}^{C_l} \mathrm{Bern}\left(g_{l,c} \mid r_{\phi,l,c}(\mathbf{x})\right). \tag{7}$$

Here, $r_{\phi,l,c}(\mathbf{x})$ is the probability of activating channel c in block l. A lightweight gating network predicts the probabilities from features available before the gated transformation:

$$\mathbf{r}_{\phi,l}(\mathbf{x}) = \sigma\left(h_{\phi,l}(\mathbf{z}_l(\mathbf{x}))\right), \tag{8}$$

where $h_{\phi,l}$ produces C_l logits and σ is the element-wise sigmoid function.

The factorization assumes conditional independence of the gate variables given the input. It does not imply that the resulting feature channels are independent.

3.3 Evidence Lower Bound

Applying Jensen's inequality yields

$$\begin{aligned} \log p_\theta(\mathbf{y} \mid \mathbf{x}) &= \log \mathbb{E}_{q_\phi(\mathbf{g}|\mathbf{x})}\left[\frac{p_\theta(\mathbf{y} \mid \mathbf{x}, \mathbf{g}) p(\mathbf{g})}{q_\phi(\mathbf{g} \mid \mathbf{x})}\right] \\ &\geq \mathbb{E}_{q_\phi(\mathbf{g}|\mathbf{x})}\left[\log p_\theta(\mathbf{y} \mid \mathbf{x}, \mathbf{g})\right] \\ &\quad - \mathrm{KL}\left(q_\phi(\mathbf{g} \mid \mathbf{x}) \,\|\, p(\mathbf{g})\right). \end{aligned} \tag{9}$$

Thus, the evidence lower bound is

$$\begin{aligned} \mathcal{L}_{\mathrm{ELBO}}(\theta, \phi; \mathbf{x}, \mathbf{y}) &= \mathbb{E}_{q_\phi(\mathbf{g}|\mathbf{x})}\left[\log p_\theta(\mathbf{y} \mid \mathbf{x}, \mathbf{g})\right] \\ &\quad - \mathrm{KL}\left(q_\phi(\mathbf{g} \mid \mathbf{x}) \,\|\, p(\mathbf{g})\right). \end{aligned} \tag{10}$$

For classification, the negative expected log-likelihood is the expected cross-entropy loss.

3.4 Binary Gate Sampling

For brevity, write $r_{l,c} = r_{\phi,l,c}(\mathbf{x})$ and define its logit

$$a_{l,c} = \log \frac{r_{l,c}}{1 - r_{l,c}}. \tag{11}$$

Sample independent noise variables

$$\epsilon_{l,c}^{(1)}, \epsilon_{l,c}^{(0)} \sim \text{Gumbel}(0, 1). \tag{12}$$

A Bernoulli gate can be sampled as

$$g_{l,c}^{\text{hard}} = \mathbf{1}\left[a_{l,c} + \epsilon_{l,c}^{(1)} - \epsilon_{l,c}^{(0)} > 0\right]. \tag{13}$$

We use the continuous relaxation

$$\widetilde{g}_{l,c} = \sigma\left(\frac{a_{l,c} + \epsilon_{l,c}^{(1)} - \epsilon_{l,c}^{(0)}}{T}\right), \tag{14}$$

where $T > 0$ is the temperature.

The straight-through gate is

$$g_{l,c}^{\text{ST}} = \text{stopgrad}\left(g_{l,c}^{\text{hard}} - \widetilde{g}_{l,c}\right) + \widetilde{g}_{l,c}. \tag{15}$$

This construction uses binary masks in the forward pass and gradients of the relaxation in the backward pass. The resulting gradient estimator is biased.

3.5 Channel Sparsity Regularization

Choose a factorized Bernoulli prior:

$$p(\mathbf{g}) = \prod_{l=1}^{L} \prod_{c=1}^{C_l} \text{Bern}(g_{l,c} \mid \pi_{l,c}). \tag{16}$$

For a single gate, with posterior activation probability r and prior activation probability π,

$$\begin{aligned} &\text{KL}\left(\text{Bern}(r) \parallel \text{Bern}(\pi)\right) \\ &= r \log \frac{r}{\pi} + (1 - r) \log \frac{1 - r}{1 - \pi} \\ &= -H(r) + r \log \frac{1 - \pi}{\pi} - \log(1 - \pi), \end{aligned} \tag{17}$$

where

$$H(r) = -r \log r - (1 - r) \log(1 - r) \tag{18}$$

is the Bernoulli entropy.

Motivated by this decomposition, define

$$\mathcal{R}_{\beta,\lambda}(\phi; \mathbf{x}) = -\beta \sum_{l,c} H(r_{l,c}) + \lambda \sum_{l,c} r_{l,c}, \tag{19}$$

where $\beta > 0$ and $\lambda \geq 0$.

The complete minimization objective is

$$\boxed{\begin{aligned} \mathcal{J}(\theta, \phi) = \mathbb{E}_{(\mathbf{x},\mathbf{y})\sim\mathcal{D}} \Bigg[&\mathbb{E}_{q_\phi(\mathbf{g}|\mathbf{x})}\left[-\log p_\theta(\mathbf{y} \mid \mathbf{x}, \mathbf{g})\right] \\ &- \beta \sum_{l,c} H(r_{l,c}) + \lambda \sum_{l,c} r_{l,c} \Bigg]. \end{aligned}} \tag{20}$$

Minimizing $-\beta H(r)$ encourages uncertain gate probabilities and discourages premature deterministic decisions. The sparsity term penalizes the expected number of active intermediate channels:

$$\mathbb{E}_{q_\phi(\mathbf{g}|\mathbf{x})}\left[\sum_{l,c} g_{l,c}\right] = \sum_{l,c} r_{l,c}. \tag{21}$$

This penalty measures expected channel count rather than exact computational cost. Channels in different blocks may have different costs because of their spatial resolutions and convolutional dimensions.

3.6 Interpretation as a Sparse Prior

For fixed β and λ, let all prior activation probabilities equal

$$\boxed{\pi = \sigma\left(-\frac{\lambda}{\beta}\right).} \tag{22}$$

Then

$$\beta \log \frac{1 - \pi}{\pi} = \lambda. \tag{23}$$

Consequently,

$$\begin{aligned} -\beta H(r) + \lambda r = \beta\, &\text{KL}\left(\text{Bern}(r) \parallel \text{Bern}(\pi)\right) \\ &+ \beta \log(1 - \pi). \end{aligned} \tag{24}$$

Summing over all gates gives

$$\begin{aligned} \mathcal{R}_{\beta,\lambda}(\phi; \mathbf{x}) = \beta\, &\text{KL}\left(q_\phi(\mathbf{g} \mid \mathbf{x}) \parallel p_\pi(\mathbf{g})\right) \\ &+ \beta K \log(1 - \pi). \end{aligned} \tag{25}$$

The final term is constant with respect to θ and ϕ when β and λ are fixed. Therefore, the proposed objective is equivalent, up to an additive constant, to the expected classification loss plus a weighted KL divergence.

When $\beta = 1$, this is the negative ELBO up to an additive constant. For general $\beta > 0$, it is a weighted KL-regularized variational objective; it need not be the negative of a standard ELBO.

Increasing λ at fixed β decreases π, strengthening the prior preference for inactive channels.

3.7 Interaction Between Entropy and Sparsity

For a single gate, consider only its regularizer:

$$R(r) = -\beta H(r) + \lambda r. \tag{26}$$

For $0 < r < 1$,

$$\frac{dR}{dr} = \beta \log \frac{r}{1 - r} + \lambda. \tag{27}$$

Setting this derivative to zero gives

$$r^* = \sigma\left(-\frac{\lambda}{\beta}\right). \tag{28}$$

Moreover,

$$\frac{d^2R}{dr^2} = \frac{\beta}{r(1 - r)} > 0, \tag{29}$$

so this is the unique minimizer of the regularizer.

Thus, the ratio λ/β determines the preferred activation probability in the absence of the prediction loss. The learned

gate probabilities also depend on how each channel affects predictive performance.

A small entropy coefficient alone does not guarantee deterministic gates, and a small sparsity coefficient alone does not guarantee that all channels remain active.

3.8 Adaptive Scheduling of the Sparsity Coefficient

For the one-shot procedure in Algorithm 1, let A_t^{ref} and A_t denote the validation accuracies of the dense reference and gated model at the same search epoch t. The controller averages the last w paired gaps:

$$\overline{\Delta}_t = \frac{1}{w} \sum_{i=t-w+1}^{t} (A_i^{\text{ref}} - A_i), \qquad t \geq w. \tag{30}$$

The coefficient λ_t is used to train epoch t. For the first W warmup epochs, and until a full window is available, $\lambda_{t+1} = \lambda_t$. Subsequently, after every epoch,

$$\lambda_{t+1} = \begin{cases} \min(\lambda_{\max}, \lambda_t e^{\eta}), & \overline{\Delta}_t \leq \delta_s, \\ \max(\lambda_{\min}, \lambda_t e^{-\eta}), & \overline{\Delta}_t > \delta_h, \\ \lambda_t, & \text{otherwise}, \end{cases} \tag{31}$$

where $0 \leq \delta_s < \delta_h$ are accuracy tolerances and $\eta > 0$ is the fixed log-step. Accuracies and tolerances are fractions. This feedback rule preserves positivity and, before clipping to $[\lambda_{\min}, \lambda_{\max}]$, applies a fixed relative change.

The rule does not guarantee convergence or satisfaction of the accuracy constraint at every epoch. In particular, validation noise and delayed responses to regularization can produce oscillations around the target accuracy drop.

For fixed β, updating λ also updates the corresponding prior:

$$\pi_t = \sigma\left(-\frac{\lambda_t}{\beta}\right). \tag{32}$$

Increasing λ_t makes the prior more sparse, while decreasing it makes the prior less sparse. This is an externally controlled prior schedule rather than optimization of a single fixed-prior ELBO throughout training.

3.9 Structural Channel Pruning

Input-dependent gates produce different masks for different inputs. Permanent channel removal requires selecting a fixed mask.

We estimate the average activation probability of each channel on a representative calibration set $\mathcal{D}_{\text{cal}}$:

$$\bar{r}_{l,c} = \frac{1}{|\mathcal{D}_{\text{cal}}|} \sum_{\mathbf{x} \in \mathcal{D}_{\text{cal}}} r_{\phi,l,c}(\mathbf{x}). \tag{33}$$

Channels satisfying

$$\bar{r}_{l,c} < \varepsilon_{\text{prune}} \tag{34}$$

are candidates for permanent removal, where $\varepsilon_{\text{prune}} \in (0, 1)$ is a pruning threshold.

For the intermediate-channel placement described above, structural pruning removes the corresponding output channels of $a_{\theta,l}$ and input channels of $b_{\theta,l}$, together with associated channel-wise parameters. The output dimensions of the residual branch and shortcut remain unchanged.

This operation assumes compatible channel connectivity, as in standard dense convolutions with channel-wise normalization. Grouped convolutions and operations that mix channels require additional structural constraints.

Pruning can be performed iteratively:

1. Train the gated network.
2. Estimate average channel activation probabilities.
3. Remove a selected set of low-probability channels.
4. Fine-tune and validate the reduced network.
5. Repeat while the desired accuracy–size trade-off remains acceptable.

Low average activation probability is a pruning heuristic, not a guarantee that a channel is unnecessary for every input. Predictive performance must therefore be evaluated after structural removal.

Binary masking alone does not reduce the cost of convolutions that have already been computed. Actual computational savings require physical channel removal or an execution backend that avoids inactive computations. Any efficiency evaluation must also account for the cost of the gating modules.

Algorithm overview. Algorithm 1 specifies the one-shot procedure with input-independent gates used for ResNet-50. Here T_s, T_r are the search and recovery budgets. Section 3.8 defines the controller, including warmup, the paired averaging window, the fixed log-step and clipping. The default automatic log-step is computed once and requires $\lfloor T_s/3 - W \rfloor \geq 1$ and $\lambda_{\min} \leq \lambda_0 < \lambda_{\text{target}} \leq \lambda_{\max}$. Numerical training settings are given in Section 4.2.4.

Let (θ_t, ϕ_t) denote the saved search checkpoint, with θ_t including the weights and corresponding BatchNorm statistics, and ϕ_t the gate logits. The operator $\text{MakeMask}_{\geq 1}(\phi_t)$ keeps all channels with $r > 0.5$ in each group; if none qualify, it keeps one channel with maximal r. This minimum can activate a channel closed in the original gated validation. For each second-half checkpoint, $\text{Eval}_{\text{val}}(\theta_t, m_t)$ measures validation accuracy with the fixed mask, without updating weights or BatchNorm statistics; no training or BatchNorm recalibration occurs during selection. $\text{Params}(m_t)$ counts the parameters of the architecture with the retained layer widths, excluding gates, without constructing each compact network; it is not an expected active-parameter count.

After selection, $\text{Compact}(\theta_{t^*}, m_{t^*})$ physically removes the discarded channels and gates, transferring surviving weights and their BatchNorm state to the fixed architecture g. Next, $\text{RecalibrateBN}(g)$ updates BatchNorm statistics on training data. Recovery uses a fresh optimizer and scheduler for T_r

epochs and returns the checkpoint with the highest validation accuracy among those epochs.

4 Experimental Setup

Experiments include iterative channel-pruning trajectories and single-pass pruning results on both model–dataset pairs.

4.1 Datasets

We evaluate MobileNetV2 on TinyImageNet-200 [7] and ResNet-50 on CIFAR-10 [6], using train/validation/test splits of 90,000/10,000/10,000 and 45,000/5,000/10,000 images, respectively. For TinyImageNet-200, the official validation set serves as the test set. Its native 64×64 images are upscaled to 224×224 inputs; CIFAR-10 inputs remain 32×32. Training uses random crops and horizontal flips with dataset-specific normalization; evaluation uses resizing to 256×256 followed by a 224×224 center crop for TinyImageNet-200, and only tensor conversion and normalization for CIFAR-10.

4.2 Implementation Details

Iterative pruning. Each cycle trains gates, updates a cumulative mask of removed channels, and trains the physically compact network without gates. Initialization and state transfer are specified for each setting below.

One-shot pruning. One search phase is followed by a single structural pruning and recovery phase; the compact architecture remains fixed during recovery. Algorithm 1 specifies the ResNet-50 setting, including checkpoint selection and weight inheritance.

The iterative MobileNetV2 and one-shot ResNet-50 configurations described below use AdamW, batch size 128 and a separate cosine learning-rate decay to zero in each phase. Weight decay is $5 \cdot 10^{-4}$, except for one-shot ResNet gate logits, which have zero weight decay.

4.2.1 MobileNetV2: iterative pruning. Each pruning cycle takes 75 epochs: 30 for search and 45 for recovery. Both search and recovery restart from ImageNet weights, with fresh gates for search; only the removed-channel set passes between cycles. The procedure stops when no channel is selected or after ten cycles. The reported run completed ten cycles (750 training epochs), producing an accuracy–size trajectory of compact recovery models.

We use MobileNetV2 [11] with its standard channel widths (2,480,072 parameters), using torchvision `IMAGENET1K_V2` weights and a new 200-way classifier. Learning rates are 10^{-4} for ImageNet-initialized weights and 10^{-3} for gates and classifier. The dense Baseline is fine-tuned for 45 epochs using the same optimizer settings.

Gate placement. We gate the hidden width of all 16 inverted residual blocks with an expansion layer, after depthwise convolution, BatchNorm and ReLU6 (ReLU clipped at 6). Each hidden gate couples one expansion filter, one depthwise filter, their BatchNorm parameters and the corresponding projection input channel. Placing the gate after normalization makes masking equivalent to physical removal; placing it before BatchNorm would allow a closed channel to become nonzero. The ten blocks with identity shortcuts also have output gates after projection BatchNorm:

$$\mathbf{x}_l = \mathbf{x}_{l-1} + \mathbf{g}_l^{\text{out}} \odot b_{\theta,l}(\mathbf{g}_l \odot a_{\theta,l}(\mathbf{x}_{l-1})). \tag{35}$$

Closing an output gate removes the corresponding projection filter and BatchNorm parameters while the shortcut preserves that channel and the block output width. Hidden and output gates follow the same objective and pruning rule: 7,104 hidden gates and 792 output gates, 7,896 in total.

Each gate has an input-independent learnable logit, initialized at $r_{l,c} \approx 0.87$ with small random perturbations. Training uses straight-through sampled masks at temperature $T = 1$; evaluation uses $r_{l,c} > 0.5$. Thus the calibration average in the Methodology section is simply $\bar{r}_{l,c} = r_{l,c}$.

Objective and normalization. Let M be the number of gate groups and C_j the $C_j > 0$ channels in group j not permanently masked in the current search phase. Hidden and output gates form separate groups. We average within groups, then across them:

$$S(r_\phi) = \frac{1}{M} \sum_{j=1}^{M} \frac{1}{C_j} \sum_{c \in C_j} r_{j,c}. \tag{36}$$

The implemented objective is

$$\mathcal{J}_{\text{impl}} = \mathcal{L}_{\text{CE}} + \lambda S(r_\phi). \tag{37}$$

Each group has equal weight, giving coefficient $\lambda/(MC_j)$ to each channel. The reported coefficients use this normalization.

Adaptive sparsity. We initialize $\lambda_0 = 10^{-4}$ and hold it fixed for three epochs. Updates then run every epoch, clipped to $[10^{-8}, 100]$. The reference is a separate dense model trained for 30 epochs, distinct from the 45-epoch Baseline. Its accuracy at epoch t is compared with the gated model's mean accuracy over the last three epochs. We set the soft and hard tolerances, δ_s and δ_h, to 2 and 4 percentage points, respectively. We increase λ for drops of at most δ_s, decrease it for drops above δ_h and hold it otherwise. The log-space step is $\eta = \log(1.25)\, 2^k$, $k \in \{0, 1, 2\}$. During increases, k rises by one (up to 2) if mean closing probability $1 - r_{l,c}$ over all gates grows by less than 0.02 since the previous update, resets for growth in $[0.02, 0.07]$ and otherwise stays unchanged; it also resets whenever λ is not increased.

Channel selection. After search, channels with $r_{l,c} < 0.95$ are considered in increasing probability order. A channel is accepted for removal if the cumulative parameter cost stays within 10% of the full gated network's parameters; candidates that do not fit are skipped, and at least one channel remains per gate group. Since gates start below 0.95, search must raise

Algorithm 1 Accuracy-guided structural channel pruning. Default controller settings are warmup $W = 10$ epochs, averaging window $w = 3$ epochs, and initial coefficient $\lambda_0 = 10^{-3}$. The empirically chosen clipping bounds are fixed at $\lambda_{\min} = 10^{-8}$ and $\lambda_{\max} = 80$. The default automatic log-step uses the fixed target $\lambda_{\text{target}} = 10$.

Require: Network f_θ; training and validation data; dense reference accuracies $\{A_t^{\text{ref}}\}_{t=1}^{T_s}$.
Require: Search and recovery budgets T_s, T_r; accuracy tolerances δ_s, δ_h.
// Search
Initialize ϕ with gate-opening probabilities close to one; $\lambda_1 \leftarrow \lambda_0$.
$\eta \leftarrow \log(\lambda_{\text{target}}/\lambda_0)/\lfloor T_s/3 - W \rfloor$.
for $t = 1, \ldots, T_s$ **do**
  Train (θ, ϕ) for one epoch with λ_t; measure validation accuracy A_t; save (θ_t, ϕ_t).
  if $t \le W$ or $t < w$ **then**
    $\lambda_{t+1} \leftarrow \lambda_t$.
  else
    Update λ_{t+1} by Eq. (31).
  end if
end for
// Checkpoint selection
$\mathcal{T} \leftarrow \{\lfloor T_s/2 \rfloor + 1, \ldots, T_s\}$.
for $t \in \mathcal{T}$ **do**
  $m_t \leftarrow \text{MakeMask}_{\ge 1}(\phi_t)$.
  $A_t^{\text{mask}} \leftarrow \text{Eval}_{\text{val}}(\theta_t, m_t)$.
  $P_t \leftarrow \text{Params}(m_t)$.
end for
$\mathcal{F} \leftarrow \{t \in \mathcal{T} : A_t^{\text{mask}} \ge A_{T_s}^{\text{ref}} - \delta_h\}$.
if $\mathcal{F} \neq \emptyset$ **then**
  $t^* \in \arg\min_{t \in \mathcal{F}} P_t$.
else
  $t^* \in \arg\max_{t \in \mathcal{T}} A_t^{\text{mask}}$.
end if
// Recovery
$g \leftarrow \text{Compact}(\theta_{t^*}, m_{t^*})$.
$g \leftarrow \text{RecalibrateBN}(g)$; initialize a fresh optimizer and scheduler.
Fine-tune g for T_r epochs.
return Best validation checkpoint of g during recovery.

their probabilities above this threshold to protect them from selection, subject to the parameter cap. Accepted channels are physically removed before recovery.

4.2.2 MobileNetV2: one-shot pruning. [TODO: insert the configuration of the one-iteration MobileNetV2 runs and any differences from the one-shot procedure.]

4.2.3 ResNet-50: iterative pruning. [TODO: insert the configuration of the iterative ResNet-50 runs and any differences from the iterative procedure.]

4.2.4 ResNet-50: one-shot pruning.

Algorithm 1 specifies the 60-epoch search and 90-epoch recovery. Recovery inherits the surviving search weights, recalibrates BatchNorm on 200 training batches and starts a fresh optimizer. We test the best validation recovery checkpoint with frozen weights and BatchNorm.

We use a CIFAR-adapted ResNet-50 [2] (23.547M parameters), with a 3×3 stride-1 stem and no max-pooling. The eight repeated runs share random initialization (seed 42); the separately trained dense model supplies only the validation reference. All parameter groups start at learning rate 10^{-3}; entropy regularization and label smoothing are disabled.

Gates and regularization. Each of the 16 bottlenecks has two independently gated internal widths, after the first and second convolutions' BatchNorm and ReLU: 32 groups, with no output gates. Unlike iterative MobileNetV2, both training and evaluation use deterministic masks $r_{j,c} > 0.5$, with straight-through training gradients and initial opening probability approximately 0.98. In Eq. (36), C_j is fixed at the group's original channel count and M at 32, the total number of gate groups; these denominators remain fixed throughout search.

Sparsity control and selection. The controller averages three paired, epoch-matched validation gaps. We set the soft and hard tolerances, δ_s and δ_h, to 0.025 and 0.05 percentage points, respectively. We use a ten-epoch warmup, $\lambda_0 = 10^{-3}$, bounds $[10^{-8}, 80]$ and the default automatic log-step, which evaluates to $\eta \approx 0.9210$ for this search configuration, without the iterative MobileNetV2 closure-rate boost. From epochs 31–60 we select the smallest feasible physical model, with a 0.001 retention floor, equivalent here to retaining at least one channel per group; fallback maximizes validation accuracy. Candidates are scored before additional BatchNorm calibration.

Earlier red/black tolerance sweeps used retention floors of 0.001/0.08. Red feasible candidates came from epochs 1–60, with fallback from 31–60; both black windows were 31–60. The additional red mean represents two selected repeats of the configuration above.

4.3 Baselines

For each architecture–dataset pair, we compare our method with a dense baseline, DepGraph [1] with random importance, AutoPruner [10], ConvNet-AIG [14], and Stochastic Depth [5].

MobileNetV2 comparisons. DepGraph uses 15 epochs of dense fine-tuning, followed by global pruning in 30 steps with target FLOPs speedups of 1.05, 1.1, 1.25, 1.5, and 2.0, and 30 epochs of post-pruning fine-tuning. The classifier is not pruned.

AutoPruner is applied to the expanded channels of the 16 blocks with expansion layers. Starting from a fine-tuned dense checkpoint, we train five models with different target sizes, using 20 epochs of channel selection followed by 25 epochs of post-pruning fine-tuning.

ConvNet-AIG uses input-dependent gates on the residual branches of the ten identity-shortcut blocks without removing weights. Models are trained from ImageNet weights for 45 epochs, with target-rate loss weight 2 and target rates $t \in \{0.01, 0.05, 0.1, 0.2, \ldots, 0.9\}$.

Stochastic Depth randomly drops the residual branches of the same ten blocks during training, with survival probabilities $p_l = 1 - \frac{l}{10}(1 - p_L)$. The expected number of parameters used per training step is $1{,}604{,}948.8 + 875{,}123.2p_L$. We set $p_L \in \{0, 0.109, 0.451, 0.680, 1\}$, corresponding to approximately 1.60M, 1.70M, 2.00M, 2.20M, and 2.48M parameters, respectively. Training comprises 30 epochs of dense fine-tuning followed by 15 epochs with Stochastic Depth. At test time, one Bernoulli mask is shared by all images in a minibatch and resampled for the next minibatch. Closed residual branches are skipped; surviving branch outputs are unscaled and BatchNorm statistics are frozen. Each minibatch executes its sampled subnetwork, while all 2.48M weights remain in storage.

ResNet-50 comparisons. On CIFAR-10, we report local reference runs under their original training protocols and parameter-count conventions. AutoPruner uses 150 epochs after a shared 150-epoch dense pretraining run; Stochastic Depth uses 200 epochs.

Stochastic Depth checkpoints are selected by full-network validation. At test time, an independent Bernoulli mask is sampled for each residual branch and minibatch (batch size 128), shared by all images in that minibatch, with frozen BatchNorm and unscaled surviving branches. We report each checkpoint's mean accuracy over five passes (seeds 42–46), without prediction ensembling.

We additionally compare with DepGraph under a matched budget of 150 training epochs per pruned model. For DepGraph, this includes any pretraining and subsequent training stages; our one-shot procedure uses 60 epochs of mask search and 90 epochs of recovery. Training our shared dense validation reference incurs an additional setup cost outside this per-model budget. Matching epoch counts does not imply equal computational cost.

4.4 Evaluation Metrics

For the main experiments, we report test top-1 accuracy at validation-selected checkpoints. For the Baseline, our method, DepGraph and AutoPruner, we count parameters in the fixed network used for testing. ConvNet-AIG reports mean active counts over test images on TinyImageNet-200 and validation images on CIFAR-10. Stochastic Depth counts are expectations over minibatch masks; test inference follows the protocols described above.

5 Results and Analysis

5.1 Classification Accuracy

5.1.1 MobileNetV2 on TinyImageNet-200.

Iterative channel pruning. Figure 1 and Table 1 show the accuracy–size trade-off of all methods. The dense Baseline reaches 71.6% accuracy with 2.48M parameters.

Mild compression. The first two cycles of our method remove 9.9% and 18.7% of the parameters, with accuracy losses of 0.4 and 1.6 percentage points, respectively. At 2.015M parameters, our model reaches 70.0% accuracy, matching DepGraph at 2.185M. The closest AutoPruner model by parameter count reaches 68.9% at 1.900M.

Stochastic Depth reaches 71.8% at a reported expected active count of 2.2M under sampled test masks. ConvNet-AIG stays within 0.3 percentage points of the Baseline for $t \geq 0.7$, with test-mean active counts of 97.7–99.2% of its parameters. Both retain all 2.48M parameters in storage.

Stronger compression. Our final model reaches 65.8% accuracy at 1.584M parameters. AutoPruner is more accurate

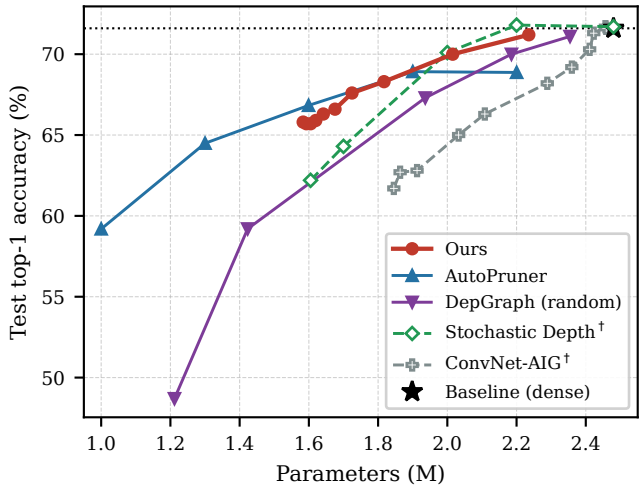


Figure 1. Test accuracy versus number of parameters of pruned MobileNetV2 on TinyImageNet-200. Each line connects the models of one run of our method (one point per pruning cycle) or of one hyperparameter sweep of a compared method. The star and the dotted line mark the dense Baseline. †: expected number of executed parameters; these methods store all 2.48M parameters.

at a similar size, reaching 66.8% at 1.599M. Below 1.5M parameters in Figure 1, only AutoPruner and DepGraph provide comparison points. AutoPruner gives the strongest observed accuracy–size trade-off in this range, reaching 64.5% at 1.300M, compared with DepGraph's 59.2% at 1.423M.

Single-pass channel pruning. The seven one-iteration MobileNetV2 models in Figure 3(a) and Table 10 span 1.262–2.235M parameters and 60.7–71.1% reported test accuracy. At nearly the same size, the one-iteration model reaches 69.5% at 2.013M, compared with 70.0% at 2.015M on the existing iterative trajectory; at 1.813M it reaches 68.4%, compared with 68.3% at 1.817M. These observations do not show a consistent advantage for either procedure.

5.1.2 ResNet-50 on CIFAR-10.

One-shot pruning. Figure 2 and Table 2 show the trade-off between accuracy and model size for ResNet-50 on CIFAR-10. The dense baseline achieves 93.93% test accuracy with 23.547M parameters.

Our 4.660M-parameter model achieves 93.31% accuracy, reducing the parameter count by 80.2% with an accuracy loss of 0.62 percentage points. The nearest DepGraph model achieves 93.07% with 4.566M parameters. Our model therefore achieves 0.24 percentage points higher accuracy with 2.1% more parameters.

Averaging two selected repeats gives 10.805M parameters and 93.78% accuracy, corresponding to 54.1% fewer parameters and an accuracy gap of 0.15 percentage points from the dense baseline. The nearest DepGraph model achieves 93.51% with 11.569M parameters.

Stochastic Depth reaches up to 94.84% accuracy but retains all 23.547M parameters in storage.

Iterative channel pruning. Three channel-pruning trajectories in Figure 3(b) contain ten models each, ordered by

Table 1. MobileNetV2 on TinyImageNet-200.

(a) Pruning trajectory

Baseline Model	Params (M)	Params (% of dense)	Test acc. (%)	Acc. change (pp)
dense	*2.480*	*100.0*	71.6	*0.0*
Ours				
Pruning cycle	**Params (M)**	**Params (% of dense)**	**Test acc. (%)**	**Acc. change (pp)**
1	*2.235*	*90.1*	71.2	*−0.4*
2	*2.015*	*81.3*	70.0	*−1.6*
3	*1.817*	*73.3*	68.3	*−3.3*
4	*1.725*	*69.5*	67.6	*−4.0*
5	*1.676*	*67.6*	66.6	*−5.0*
6	*1.642*	*66.2*	66.3	*−5.3*
7	*1.620*	*65.3*	65.9	*−5.7*
8	*1.605*	*64.7*	65.7	*−5.9*
9	*1.593*	*64.2*	65.7	*−5.9*
10 (final)	*1.584*	*63.8*	**65.8**	*−5.8*

(b) Comparison at similar model sizes

Size (M)	Ours		AutoPruner		DepGraph	
	Acc. (%)	Size (M)	Acc. (%)	Size (M)	Acc. (%)	Size (M)
≈1.6	65.7 *cycle 8*	*1.605*	66.8 *r=0.5125*	*1.599*	–	–
≈1.9	68.3 *cycle 3*	*1.817*	68.9 *r=0.6792*	*1.900*	67.3 *×1.25*	*1.936*
≈2.0	70.0 *cycle 2*	*2.015*	68.9 *r=0.6792*	*1.900*	67.3 *×1.25*	*1.936*
≈2.2	71.2 *cycle 1*	*2.235*	68.9 *r=0.8452*	*2.200*	70.0 *×1.1*	*2.185*

Cells (b,c): accuracy (%); *params (M), setting.*

Nearest size within ±5%; no interpolation. Repeated entries reuse a model. *r*: target channel keep ratio; ×: target FLOPs speedup.

(c) Active-count results †

Method	≈ 1.6M	≈ 1.9M	≈ 2.0M	≈ 2.2M
Stoch. Depth	62.2 *(1.605)* p_L=0.000	–	70.1 *(2.000)* p_L=0.451	71.8 *(2.200)* p_L=0.680
ConvNet-AIG	–	62.8 *(1.913)* *t=0.1*	65.0 *(2.033)* *t=0.2*	68.2 *(2.289)* *t=0.4*

†: both store 2.48M parameters. AIG reports test means; SD reports expected active counts under sampled test masks. Active counts are not storage or latency measurements.

One run per setting; the ten Ours points share one run. Training budgets differ.

In (a), shares and changes use the dense Baseline (2,480,072 parameters); pp denotes percentage points.

Full sweeps: Table 9.

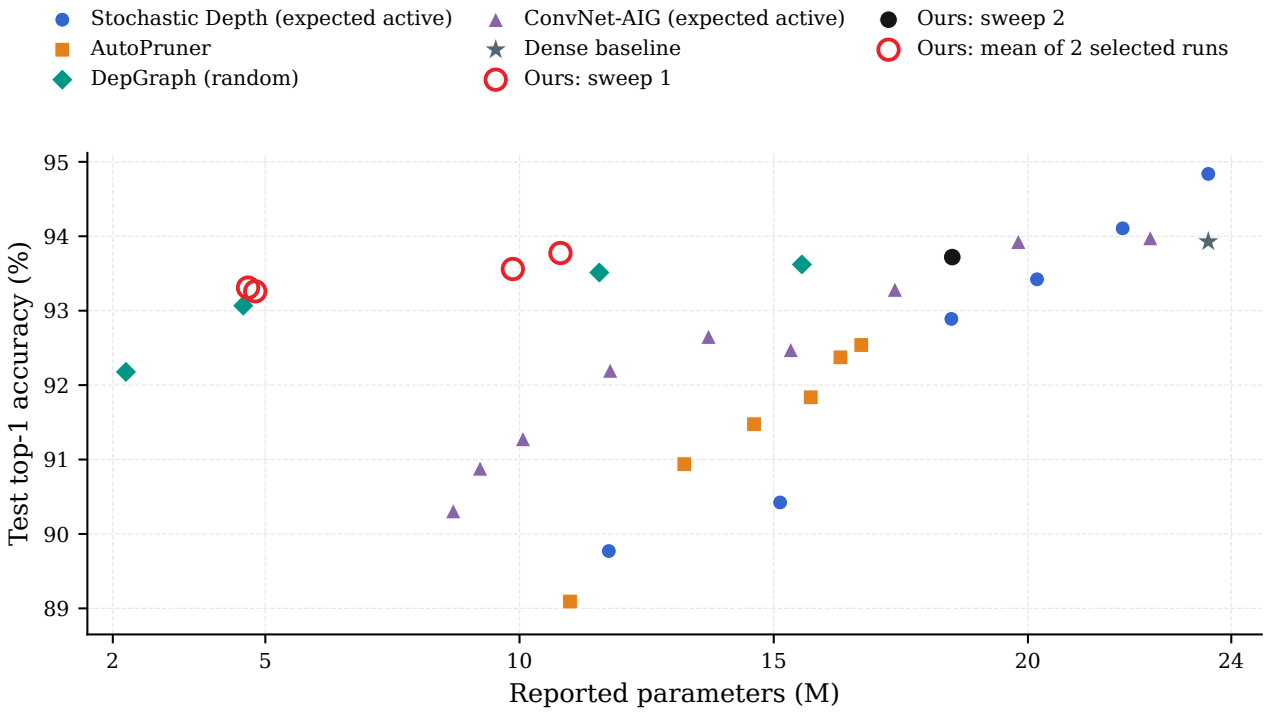


Figure 2. ResNet-50 on CIFAR-10. Hollow red circles show three selected deployments from sweep 1 and the coordinate-wise mean of two selected runs from eight repeats of one fixed pruning configuration. The filled black circle shows one selected deployment from sweep 2. All 26 comparison points are retained. Our method, DepGraph, AutoPruner and the dense baseline use physical parameter counts. ConvNet-AIG uses active parameters averaged over validation images; Stochastic Depth uses the expected active count under sampled inference. Both retain the full backbone weights. Training protocols and historical accuracy aggregation differ.

cycle rather than by notebook row order. The series labeled `thr0.7` reaches 92.1% at 5.325M parameters before declining to 87.8% at 3.220M in its final model. The `thr0.1` series ends at 91.4% with 5.787M parameters. The more conservative series labeled `param_fraction_0.05` retains 15.475M parameters with 93.2% accuracy at the end. Complete trajectories are given in Table 11.

Block pruning. Four additional block-pruning models span 15.266–18.974M parameters and 92.80–93.27% reported accuracy. Their final and preceding recovery models have the same parameter count but different accuracies; they are separate checkpoints, not seed repetitions.

5.2 Computational Efficiency

5.3 Effect of Adaptive λ Scheduling

5.4 Iterative Pruning Analysis

MobileNetV2 on TinyImageNet-200. Our run yields one model per cycle (Table 1). Most parameters are removed in the first cycles: after three cycles it has removed 26.7% of the dense parameters. Between cycle 5 and the final model the size changes only from 67.6% to 63.8% of the dense model (accuracy from 66.6% to 65.8%). These are single-run observations.

5.5 Ablation Studies

We report six diagnostic studies on ResNet-50/CIFAR-10. Test accuracies are measured on the official test set after validation-based checkpoint selection. The comparisons are exploratory: their outcomes informed subsequent experiments. Each comparison has its own architecture and training protocol, as specified below. Figure 4 summarizes these studies alongside the main accuracy–size comparison.

Weight inheritance. We fix a pruned architecture with 17,321,706 parameters and compare inherited and randomly initialized weights. Both branches use the same 60-epoch search result followed by 90 epochs of recovery. Inherited weights reach 93.77% test accuracy, compared with 92.95% from random initialization, a difference of 0.82 percentage points (Table 3). This result concerns one selected architecture and one search result; it does not establish an advantage across architectures or training seeds.

Table 2. Test top-1 results for all 31 ResNet-50/CIFAR-10 points in Figure 2. Params: reported parameter count (M) and its share of the dense model (23,547,338 parameters, %). Δ: accuracy difference from the Baseline in percentage points. S1 and S2 denote our two sweeps; the following pair gives δ_s/δ_h in percentage points. Mean averages both coordinates of two selected runs from eight same-configuration, same-seed repeats; it is not one deployed network. DepGraph settings are target FLOPs speedups; k is AutoPruner's target keep ratio, p_L the final survival probability, and t the ConvNet-AIG target rate. †: expected active parameters under stochastic inference; ‡: active parameters averaged over validation images. Both marked methods retain the full backbone weights. Training budgets and historical accuracy aggregation differ.

Setting	Params (M)	%	Acc. (%)	Δ
Baseline				
dense	23.547	100.0	93.93	0.00
Ours (selected)				
S1: 0.025/0.05	4.660	19.8	93.31	−0.62
S1: 0.25/0.5	4.801	20.4	93.26	−0.67
S1: 5/10	9.868	41.9	93.56	−0.37
Mean ($n = 2$)	10.805	45.9	93.78	−0.15
S2: 0.25/0.5	18.512	78.6	93.72	−0.21
DepGraph (random importance)				
×1.5	15.554	66.1	93.62	−0.31
×2.0	11.569	49.1	93.51	−0.42
×5.0	4.566	19.4	93.07	−0.86
×10.0	2.258	9.6	92.18	−1.75
AutoPruner				
$k = 0.3$	10.993	46.7	89.09	−4.84
$k = 0.5$	13.242	56.2	90.94	−2.99
$k = 0.6$	14.612	62.1	91.47	−2.45
$k = 0.7$	15.728	66.8	91.84	−2.09
$k = 0.8$	16.312	69.3	92.37	−1.55
$k = 0.9$	16.722	71.0	92.54	−1.39
Stochastic Depth				
$p_L = 0.3$	$11.756^{\dagger}$	49.9	89.77	−4.16
$p_L = 0.5$	$15.125^{\dagger}$	64.2	90.42	−3.50
$p_L = 0.7$	$18.494^{\dagger}$	78.5	92.89	−1.04
$p_L = 0.8$	$20.179^{\dagger}$	85.7	93.42	−0.51
$p_L = 0.9$	$21.863^{\dagger}$	92.8	94.11	+0.18
$p_L = 1.0$	$23.547^{\dagger}$	100.0	94.84	+0.91
ConvNet-AIG				
$t = 0.1$	$8.696^{\ddagger}$	36.9	90.31	−3.62
$t = 0.2$	$9.224^{\ddagger}$	39.2	90.88	−3.05
$t = 0.3$	$10.068^{\ddagger}$	42.8	91.28	−2.65
$t = 0.4$	$11.783^{\ddagger}$	50.0	92.20	−1.73
$t = 0.5$	$13.717^{\ddagger}$	58.3	92.65	−1.28
$t = 0.6$	$15.335^{\ddagger}$	65.1	92.47	−1.45
$t = 0.7$	$17.383^{\ddagger}$	73.8	93.29	−0.64
$t = 0.8$	$19.811^{\ddagger}$	84.1	93.93	0.00
$t = 0.9$	$22.406^{\ddagger}$	95.2	93.98	+0.05

Table 3. Recovery initialization for a fixed architecture. Accuracy is reported in percent.

Initialization	Validation	Test
Inherited weights	94.38	93.77
Random weights	93.82	92.95

Table 4. Test accuracy (%) for state-transfer policies. The mean is computed over the two recovery repetitions.

AdamW	Cosine	Rep. 1	Rep. 2	Mean
Fresh	Restarted	93.18	93.10	93.140
Transferred	Restarted	93.15	93.35	93.250
Transferred	Continued	92.93	93.02	92.975

Optimizer and scheduler state transfer. For a separate, fixed 10,435,943-parameter architecture, we compare three recovery policies from a common search checkpoint, with a 60/90 search/recovery split and two recovery repetitions per policy. With a restarted cosine schedule, transferring AdamW states gives 93.250% mean test accuracy, compared with 93.140% using fresh states (Table 4). This 0.11-point difference changes sign between repetitions. Continuing the cosine schedule with transferred AdamW states gives 92.975%, 0.275 points below restarting it. Two recovery repetitions do not support a strong statistical conclusion.

Entropy regularization at similar sizes. We compare two runs with $\beta = 0$ and $\beta = 0.3$, retrospectively selected for their similar physical parameter counts. Both use 60 epochs of search, 90 epochs of recovery, adaptive λ, and the same accuracy tolerances and selector based on the expected open count. The $\beta = 0$ model reaches 93.48% test accuracy at 7.034M parameters; the $\beta = 0.3$ model reaches 93.42% at 7.073M (Table 5). The observed difference is 0.06 percentage points. These single-seed runs come from different series and code revisions and have different architectures and MAC counts, so the pair does not isolate the effect of entropy regularization.

Comparison with random channel selection. A separate later adaptive run reaches 93.72% test accuracy with 12.728M parameters, compared with 93.36% at 12.984M for a historical random-selection control (Table 6). Both models

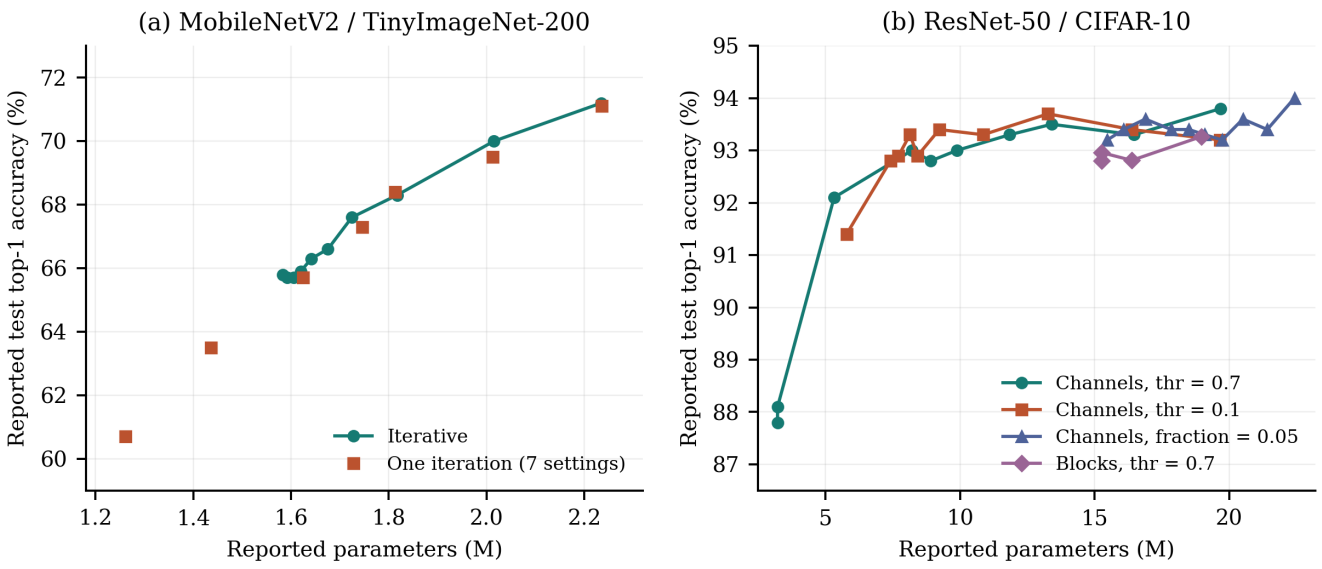


Figure 3. Additional notebook-reported pruning results. (a) Seven one-iteration MobileNetV2 settings and the existing iterative trajectory. (b) Three iterative channel-pruning trajectories and one block-pruning trajectory for ResNet-50. Lines connect successive recorded cycles; the final model is placed last. Squares in (a) are separate settings, not cycles. Counts are reported model parameters, not FLOPs or active-count estimates. No uncertainty bars or matched-budget claim are implied.

Table 5. Entropy comparison using physical deployment sizes. Accuracy is in percent; M and G denote millions and billions.

β	Params (M)	MACs (G)	Validation	Test
0	7.034	0.538	93.92	93.48
0.3	7.073	0.503	93.96	93.42

Table 6. Historical random control and the later adaptive recipe. Sizes describe physical models; test accuracy is in percent.

Method	Params (M)	MACs (G)	Test
Random selection	12.984	0.692	93.36
Our adaptive recipe	12.728	1.108	93.72

use 150 training epochs, but the adaptive run uses a single 60-epoch search followed by 90 epochs of recovery, whereas the random control uses an earlier iterative protocol with fixed λ. The random model also has fewer MACs: 0.692G versus 1.108G. The 0.36-point accuracy difference therefore compares complete training recipes and does not isolate channel ranking or adaptive sparsity scheduling.

Internal and output channel gates. An earlier experiment uses fixed $\lambda = 0.001$ and 150 training epochs per model to compare gates on residual-branch outputs with

Table 7. Gate placement in the early fixed-λ experiment. Accuracy is in percent and parameter counts are physical.

Gate location	Params (M)	Validation	Test
Branch output	20.189	93.88	93.16
Bottleneck internal	12.983	94.04	93.23

gates on internal bottleneck channels. Internal gating produces 12.983M parameters at 93.23% test accuracy, compared with 20.189M at 93.16% for output gating (Table 7). These runs demonstrate a smaller model at similar accuracy, but their pruning budgets and resulting architectures differ. They do not isolate gate placement at a fixed size or evaluate adaptive scheduling.

Label smoothing during recovery. We additionally report five historical runs with label smoothing $\varepsilon_{\mathrm{LS}} = 0.05$ during recovery (Table 8). Search uses ordinary cross-entropy without label smoothing. Each run uses 60 epochs of adaptive mask search followed by 90 epochs of recovery, with seed 42, a minimum channel-retention ratio of 0.08 and entropy regularization disabled. The five runs use different accuracy tolerances and produce different pruned architectures. Among these five runs, the highest test accuracy is **94.01%** with **11.934M** parameters; a smaller model reaches 93.35% with 5.086M parameters.

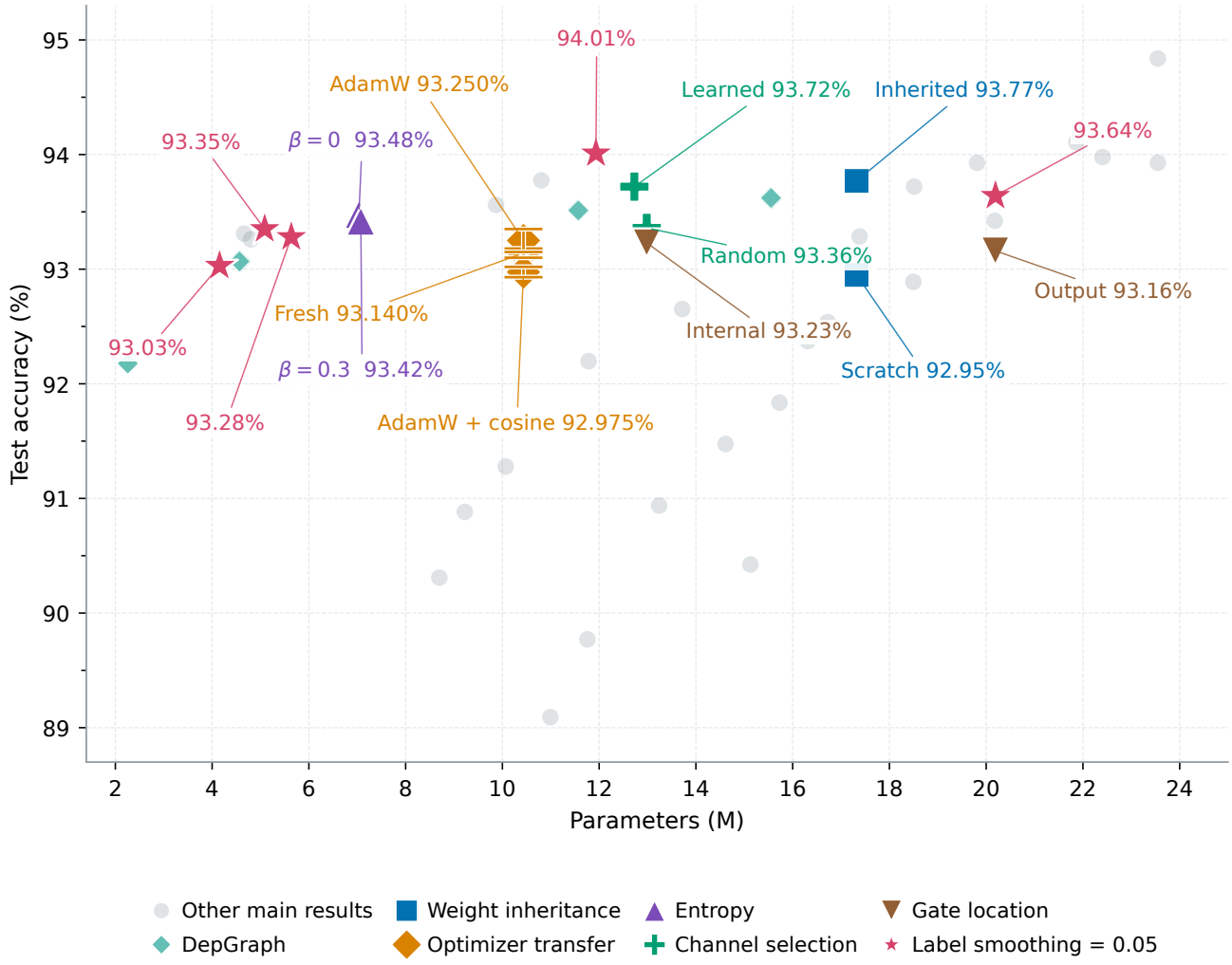


Figure 4. Diagnostic comparisons for ResNet-50 on CIFAR-10. Colored markers show the six studies reported in this section; faded markers reproduce Figure 2, with DepGraph highlighted. Optimizer-transfer markers are means of two recovery repetitions; bars span their minimum and maximum, not confidence intervals. Parameter counts are physical for the ablations, while background ConvNet-AIG and Stochastic Depth use active and expected-active counts, respectively. The historical studies use different protocols and are exploratory.

The search learning-rate floor is $6.6443 \cdot 10^{-4}$, compared with zero in the main series; there is also no separate BatchNorm recalibration before recovery. The results therefore characterize the complete training recipe; they do not isolate the effect of label smoothing alone.

Table 8. Historical runs with label smoothing 0.05 during recovery. Parameter counts are physical; accuracy is measured on the official CIFAR-10 test set.

Parameters (M)	Test accuracy (%)
4.154	93.03
5.086	93.35
5.637	93.28
11.934	**94.01**
20.192	93.64

6 Discussion

6.1 Limitations

6.2 Future Work

7 Conclusion

Conclusion text.

Acknowledgments

Acknowledgments text.

References

[1] Gongfan Fang, Xinyin Ma, Mingli Song, Michael Bi Mi, and Xinchao Wang. 2023. DepGraph: Towards Any Structural Pruning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 16091–16101. https://openaccess.thecvf.com/content/CVPR2023/html/Fang_DepGraph_Towards_Any_Structural_Pruning_CVPR_2023_paper.html

[2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Deep Residual Learning for Image Recognition. arXiv:1512.03385. https://arxiv.org/abs/1512.03385

[3] Yihui He, Ji Lin, Zhijian Liu, Hanrui Wang, Li-Jia Li, and Song Han. 2018. AMC: AutoML for Model Compression and Acceleration on Mobile Devices. In *Proceedings of the European Conference on Computer Vision*. 784–800. https://openaccess.thecvf.com/content_ECCV_2018/html/Yihui_He_AMC_Automated_Model_ECCV_2018_paper.html

[4] Charles Herrmann, Richard Strong Bowen, and Ramin Zabih. 2020. Channel Selection Using Gumbel Softmax. In *Computer Vision – ECCV 2020*. 241–257. doi:10.1007/978-3-030-58583-9_15

[5] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q. Weinberger. 2016. Deep Networks with Stochastic Depth. In *Computer Vision – ECCV 2016*. https://arxiv.org/abs/1603.09382

[6] Alex Krizhevsky. 2009. *Learning Multiple Layers of Features from Tiny Images*. Technical Report. University of Toronto. https://cave.cs.toronto.edu/kriz/learning-features-2009-TR.pdf

[7] Ya Le and Xuan Yang. 2015. Tiny ImageNet Visual Recognition Challenge. CS231N project report, Stanford University. https://cs231n.stanford.edu/reports/2015/pdfs/yle_project.pdf

[8] Zhuang Liu, Jianguo Li, Zhiqiang Shen, Gao Huang, Shoumeng Yan, and Changshui Zhang. 2017. Learning Efficient Convolutional Networks Through Network Slimming. In *Proceedings of the IEEE International Conference on Computer Vision*. 2736–2744. https://openaccess.thecvf.com/content_iccv_2017/html/Liu_Learning_Efficient_Convolutional_ICCV_2017_paper.html

[9] Christos Louizos, Max Welling, and Diederik P. Kingma. 2018. Learning Sparse Neural Networks through L_0 Regularization. In *International Conference on Learning Representations*. https://openreview.net/forum?id=H1Y8hhg0b

[10] Jian-Hao Luo and Jianxin Wu. 2020. AutoPruner: An End-to-End Trainable Filter Pruning Method for Efficient Deep Model Inference. *Pattern Recognition* 107 (2020), 107461. doi:10.1016/j.patcog.2020.107461

[11] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2018. MobileNetV2: Inverted Residuals and Linear Bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 4510–4520. https://arxiv.org/abs/1801.04381

[12] Simone Scardapane, Danilo Comminiello, Amir Hussain, and Aurelio Uncini. 2017. Group Sparse Regularization for Deep Neural Networks. *Neurocomputing* 241 (2017), 81–89. doi:10.1016/j.neucom.2017.02.029

[13] Yang Sui, Miao Yin, Yi Xie, Huy Phan, Saman Zonouz, and Bo Yuan. 2021. CHIP: CHannel Independence-based Pruning for Compact Neural Networks. In *Advances in Neural Information Processing Systems*, Vol. 34. https://arxiv.org/abs/2110.13981

[14] Andreas Veit and Serge Belongie. 2018. Convolutional Networks with Adaptive Inference Graphs. In *Proceedings of the European Conference on Computer Vision*. 3–18. https://openaccess.thecvf.com/content_ECCV_2018/html/Andreas_Veit_Convolutional_Networks_with_ECCV_2018_paper.html

[15] Yutaro Yamada, Ofir Lindenbaum, Sahand Negahban, and Yuval Kluger. 2020. Feature Selection using Stochastic Gates. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*. 10648–10659. https://proceedings.mlr.press/v119/yamada20a.html

[16] Zhuangwei Zhuang, Mingkui Tan, Bohan Zhuang, Jing Liu, Yong Guo, Qingyao Wu, Junzhou Huang, and Jinhui Zhu. 2018. Discrimination-aware Channel Pruning for Deep Neural Networks. In *Advances in Neural Information Processing Systems*, Vol. 31. https://arxiv.org/abs/1810.11809

A Additional Experiments

A.1 Detailed Results

Table 9 retains all MobileNetV2 settings and results underlying Figure 1; Table 1 highlights comparisons at similar sizes.

A.2 Hyperparameter Sensitivity

Received Submission date; revised Revision date; accepted Acceptance date

Table 9. Complete MobileNetV2 results on TinyImageNet-200. Shares and accuracy changes are relative to the dense Baseline (2,480,072 parameters); pp denotes percentage points. †: test-mean active parameters for ConvNet-AIG; analytical expected active parameters for Stochastic Depth. Both store 2.48M parameters; Stochastic Depth skips closed residual branches at test time.

Baseline				
Model	**Params (M)**	**Params (% of dense)**	**Test acc. (%)**	**Acc. change (pp)**
dense	2.480	100.0	71.6	0.0
Ours				
Pruning cycle	**Params (M)**	**Params (% of dense)**	**Test acc. (%)**	**Acc. change (pp)**
1	2.235	90.1	71.2	−0.4
2	2.015	81.3	70.0	−1.6
3	1.817	73.3	68.3	−3.3
4	1.725	69.5	67.6	−4.0
5	1.676	67.6	66.6	−5.0
6	1.642	66.2	66.3	−5.3
7	1.620	65.3	65.9	−5.7
8	1.605	64.7	65.7	−5.9
9	1.593	64.2	65.7	−5.9
10 (final)	**1.584**	**63.8**	**65.8**	**−5.8**
DepGraph (random importance)				
Target FLOPs speedup	**Params (M)**	**Params (% of dense)**	**Test acc. (%)**	**Acc. change (pp)**
×1.05	2.354	94.9	71.1	−0.5
×1.1	2.185	88.1	70.0	−1.6
×1.25	1.936	78.1	67.3	−4.3
×1.5	1.423	57.4	59.2	−12.4
×2.0	1.212	48.9	48.7	−22.9
AutoPruner				
Target channel keep ratio	**Params (M)**	**Params (% of dense)**	**Test acc. (%)**	**Acc. change (pp)**
0.1809	1.000	40.3	59.2	−12.4
0.3469	1.300	52.4	64.5	−7.1
0.5125	1.599	64.5	66.8	−4.8
0.6792	1.900	76.6	68.9	−2.7
0.8452	2.200	88.7	68.9	−2.7
Stochastic Depth †				
Survival prob. p_L	**Active params (M)**	**Params (% of dense)**	**Test acc. (%)**	**Acc. change (pp)**
0.000	1.605	64.7	62.2	−9.4
0.109	1.700	68.5	64.3	−7.3
0.451	2.000	80.6	70.1	−1.5
0.680	2.200	88.7	71.8	+0.2
1.000	2.480	100.0	71.7	+0.1
ConvNet-AIG †				
Target rate t	**Active params (M)**	**Params (% of dense)**	**Test acc. (%)**	**Acc. change (pp)**
0.01	1.845	74.4	61.7	−9.9
0.05	1.864	75.1	62.7	−8.9
0.1	1.913	77.1	62.8	−8.8
0.2	2.033	82.0	65.0	−6.6
0.3	2.108	85.0	66.3	−5.3
0.4	2.289	92.3	68.2	−3.4
0.5	2.360	95.1	69.2	−2.4
0.6	2.411	97.2	70.3	−1.3
0.7	2.423	97.7	71.3	−0.3
0.8	2.457	99.1	71.7	+0.1
0.9	2.459	99.2	71.6	0.0

Table 10. Seven one-iteration MobileNetV2 models from the additional notebook. Settings reproduce `thr`/`max_param` in run names; the latter is not the achieved parameter share.

`thr`	`max_param`	Params (M)	Test (%)
0.99	0.6	1.262	60.7
0.99	0.5	1.436	63.5
0.99	0.4	1.624	65.7
0.95	0.4	1.745	67.3
0.95	0.3	1.813	68.4
0.95	0.2	2.013	69.5
0.95	0.1	2.235	71.1

Table 11. Additional ResNet-50 trajectories. Channels: run-name settings `thr0.7`, `thr0.1` and `param_fraction_0.05`; blocks: `thr0.7`. Cycles retain zero-based source indices. Final denotes the separately recorded final model. Channel accuracies are supplied to 0.1 percentage point; block values retain historical aggregation. Rows are checkpoints, not independent repeats.

	Channels: `thr0.7`		Channels: `thr0.1`		Channels: fraction 0.05	
Cycle	Params (M)	Test (%)	Params (M)	Test (%)	Params (M)	Test (%)
0	19.680	93.8	19.673	93.2	22.427	94.0
1	16.470	93.3	16.376	93.4	21.418	93.4
2	13.404	93.5	13.275	93.7	20.514	93.6
3	11.845	93.3	10.856	93.3	19.746	93.2
4	9.881	93.0	9.233	93.4	19.097	93.3
5	8.917	92.8	8.426	92.9	18.508	93.4
6	8.214	93.0	8.141	93.3	17.846	93.4
7	5.325	92.1	7.718	92.9	16.887	93.6
8	3.224	88.1	7.418	92.8	16.082	93.4
Final	3.220	87.8	5.787	91.4	15.475	93.2
Block pruning: `thr0.7`						
0	18.974	93.27				
1	16.384	92.81				
2	15.266	92.96				
Final	15.266	92.80				

Table 12. Earlier comparison series preserved from the additional notebook, separately from the main benchmark series. MobileNetV2 Stochastic Depth (SD): reported expected active counts; all 2.480M weights remain stored. ResNet-50 DepGraph: reported physical counts and target FLOPs speedups. DepGraph values retain only the precision of the saved notebook display. Their executed protocols are not established here.

Series / setting	Params (M)	Test (%)
MobileNetV2 / TinyImageNet-200 (SD)		
SD	1.605	64.7
SD	1.700	65.3
SD	2.000	69.0
SD	2.200	70.5
SD	2.480	70.9
ResNet-50 / CIFAR-10 (DepGraph)		
×5	4.581	92.68
×2	11.278	93.43
×10	2.247	92.07
×1.5	15.015	93.37